IF2211 STRATEGI ALGORITMA

LAPORAN TUGAS KECIL 2

Voxelization 3D menggunakan Octree dengan Algoritma *Divide and Conquer*

Dibuat oleh:

13524009 - Mikhael Benrael Tampubolon

PROGRAM STUDI TEKNIK INFORMATIKA

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

JL. GANESA 10, BANDUNG 40132

2025

A. Penjelasan Algoritma Divide and Conquer

Program mengimplementasikan voxelization permukaan objek 3D yang direpresentasikan sebagai kumpulan segitiga dari file .obj. Strategi yang digunakan adalah Divide and Conquer berbasis struktur data Octree, yaitu ruang 3D dibagi secara rekursif menjadi 8 sub-kubus hingga kedalaman maksimum yang ditentukan, dan setiap oktan pada kedalaman tersebut yang masih mengandung segitiga dicatat sebagai sebuah voxel.

1. Membaca Input: read_obj()

Sebelum algoritma utama dijalankan, program membaca file .obj baris per baris menggunakan fungsi read_obj(). Baris berawalan 'v' dibaca sebagai vertex (x, y, z). Baris berawalan 'f' dibaca sebagai face poligon, lalu didekomposisi menjadi segitiga-segitiga melalui fan triangulation.

```
while (fgets(line, sizeof(line), f)) {
    if (strncmp(line, "v ", 2) == 0) {
        Vector3 p;
        if (sscanf(line + 2, "%lf %lf %lf", &p.x, &p.y, &p.z) == 3) {
            if (num_points >= capacity_points) {
                capacity_points = capacity_points == 0 ? 1024 : capacity_points * 2;
                points = realloc(points, capacity_points * sizeof(Vector3));
            }
            points[num_points++] = p;
        }
    } else if (strncmp(line, "f ", 2) == 0) {
        int v[10];
        int parsed = 0;
        char *token = strtok(line + 2, " \r\n");
```

2. Algoritma Inti: process_space()

Fungsi rekursif yang mengimplementasikan pola Divide and Conquer. Fungsi ini menerima sebuah bounding box (ruang saat ini), level kedalaman saat ini, level maksimum, serta daftar indeks segitiga yang aktif dengan ruang tersebut.

- **Basis**

  Fungsi rekursif yang mengimplementasikan pola Divide and Conquer. Fungsi ini menerima sebuah bounding box (ruang saat ini), level kedalaman saat ini, level maksimum, serta daftar indeks segitiga yang aktif (berpotensi bersinggungan) dengan ruang tersebut.

```
if (active_count == 0) {
    pruned_count[current_level]++;
    return;
}

if (current_level == max_level) {
    add_voxel(box);
    return;
}
```

- **Divide: Membagi ruang mnejadi 8 oktan**

  Jika belum mencapai basis rekursi, ruang saat ini dibagi menjadi 8 oktan dengan membelah box pada titik tengahnya di ketiga sumbu (x, y, z).

```
Vector3 center = {
    (box.min.x + box.max.x) / 2.0,
    (box.min.y + box.max.y) / 2.0,
    (box.min.z + box.max.z) / 2.0
};

BoundingBox children[8];
for (int i = 0; i < 8; i++) {
    children[i].min.x = (i & 1) ? center.x : box.min.x;
    children[i].max.x = (i & 1) ? box.max.x : center.x;
    children[i].min.y = (i & 2) ? center.y : box.min.y;
    children[i].max.y = (i & 2) ? box.max.y : center.y;
    children[i].min.z = (i & 4) ? center.z : box.min.z;
    children[i].max.z = (i & 4) ? box.max.z : center.z;
```

- **Conquer: Filter segitiga dan rekursi**

  Untuk setiap oktan anak, program menyaring segitiga yang benar-benar bersinggungan dengan oktan tersebut menggunakan fungsi check_intersection(). Hanya segitiga yang lolos yang diteruskan ke pemanggilan rekursif berikutnya.

```c
    int *buffer = malloc(active_count * sizeof(int));
    int pass_count = 0;

    for (int j = 0; j < active_count; j++) {
        if (check_intersection(active_segitiga[j], children[i])) {
            buffer[pass_count++] = active_segitiga[j];
        }
    }
    process_space(children[i], current_level + 1, max_level, buffer, pass_count);
    free(buffer);
}
```

- **check_intersection()**

  Fungsi untuk menentukan apakah sebuah segitiga bersinggungan dengan Axis-Aligned Bounding Box (AABB) menggunakan Separating Axis Theorem (SAT). Jika ditemukan satu sumbu yang memisahkan proyeksi segitiga dan kotak, keduanya dinyatakan tidak bersinggungan dan fungsi langsung return false.

  Tiga kelompok sumbu yang diuji:

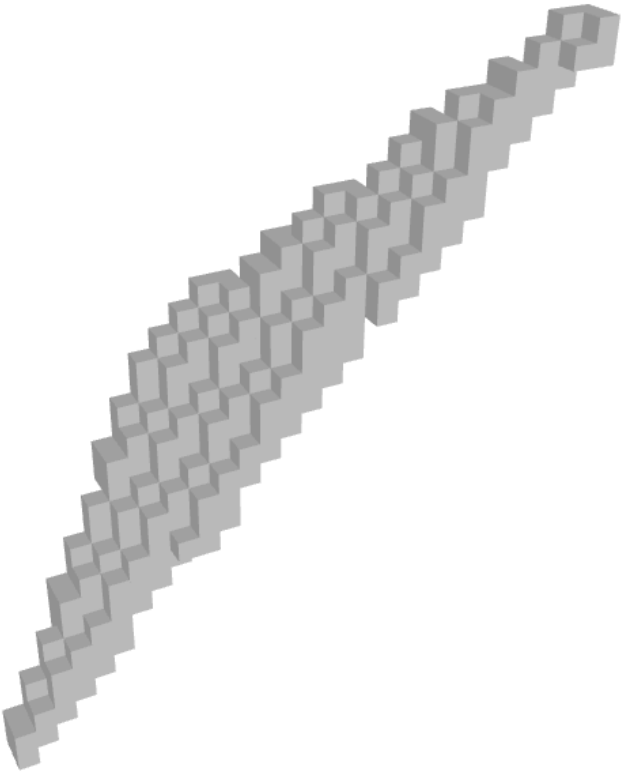
  - Sumbu AABB (x, y, z): uji overlap bounding box segitiga dengan kotak
  - Sumbu normal segitiga: proyeksikan kotak ke bidang segitiga dan cek apakah menembus.
  - 9 sumbu cross product: persilangan 3 sisi segitiga dengan 3 sumbu kotak (X, Y, Z).

```
    Vector3 edges[3] = {
        vec_sub(t->b, t->a),
        vec_sub(t->c, t->b),
        vec_sub(t->a, t->c)
    };

    Vector3 box_axes[3] = {
        {1,0,0},
        {0,1,0},
        {0,0,1}
    };

    for (int ei = 0; ei < 3; ei++) {
        for (int ai = 0; ai < 3; ai++) {
            Vector3 axis = vec_cross(edges[ei], box_axes[ai]);
            if (vec_dot(axis, axis) >= 1e-12) {
                double p1 = vec_dot(t->a, axis);
                double p2 = vec_dot(t->b, axis);
                double p3 = vec_dot(t->c, axis);

                double tmin = fmin(p1, fmin(p2, p3));
                double tmax = fmax(p1, fmax(p2, p3));

                double bmin, bmax;
                project_box(box, axis, &bmin, &bmax);

                if (tmax < bmin || bmax < tmin) return false;
            }
        }
    }
    return true;
}
```

B. Test Case

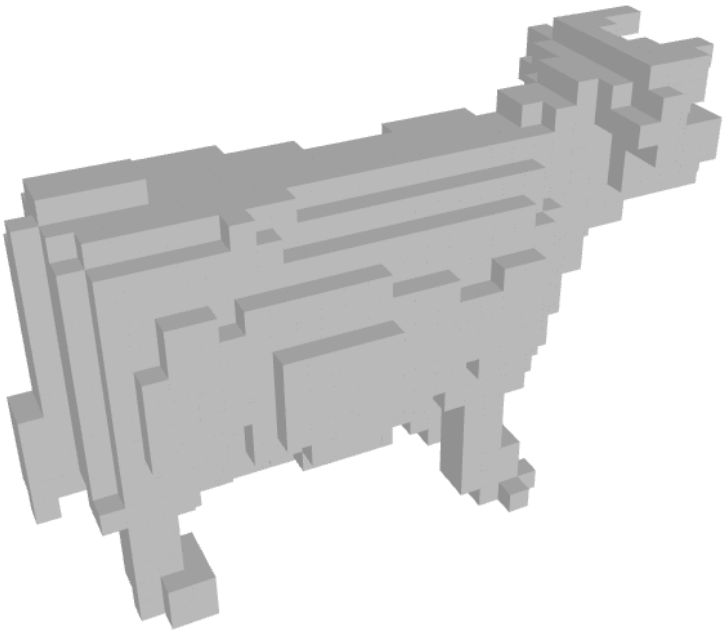
1. Line.obj kedalaman 5:

```
PS C:\Users\Asus\OneDrive\Documents\Tucil2_STIMA_13524009> gcc -O2 -o bin/voxelize src/main.c -lm
PS C:\Users\Asus\OneDrive\Documents\Tucil2_STIMA_13524009> bin/voxelize test/line.obj 5

Banyaknya voxel yang terbentuk: 287
Banyaknya vertex yang terbentuk: 2296
Banyaknya faces yang terbentuk: 1722

Statistik node octree yang terbentuk:
1 : 8
2 : 24
3 : 56
4 : 216
5 : 672

Statistik node yang tidak perlu ditelusuri:
1 : 5
2 : 17
3 : 29
4 : 132
5 : 385

Kedalaman octree: 5
Lama waktu program berjalan: 0.000 s
Path file .obj disimpan: test/line.obj_hasil.obj
```



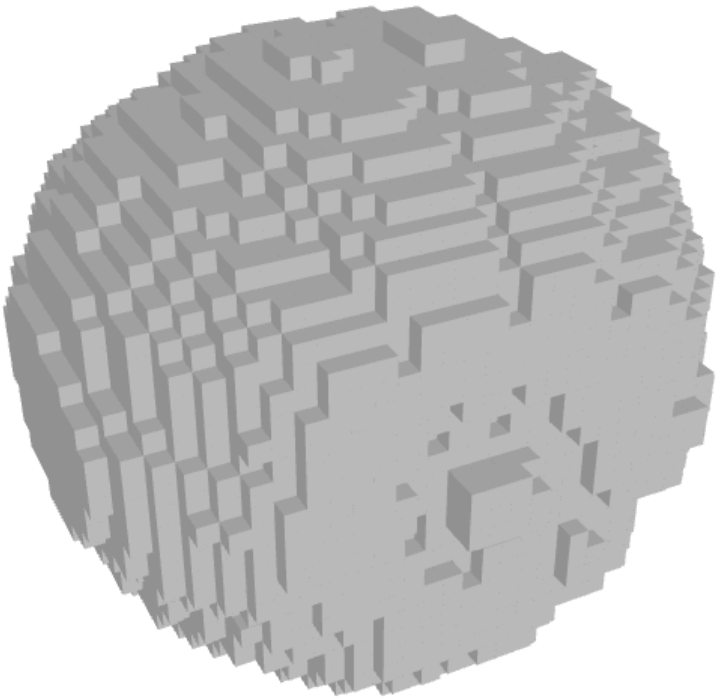
2. Cow.obj kedalaman 5:

```
PS C:\Users\Asus\OneDrive\Documents\Tucil2_STIMA_13524009> gcc -O2 -o bin/voxelize src/main.c -lm
PS C:\Users\Asus\OneDrive\Documents\Tucil2_STIMA_13524009> bin/voxelize test/cow.obj 5

Banyaknya voxel yang terbentuk: 1361
Banyaknya vertex yang terbentuk: 10888
Banyaknya faces yang terbentuk: 8166

Statistik node octree yang terbentuk:
1 : 8
2 : 32
3 : 152
4 : 696
5 : 2728

Statistik node yang tidak perlu ditelusuri:
1 : 4
2 : 13
3 : 65
4 : 355
5 : 1367

Kedalaman octree: 5
Lama waktu program berjalan: 0.024 s
Path file .obj disimpan: test/cow.obj_hasil.obj
```



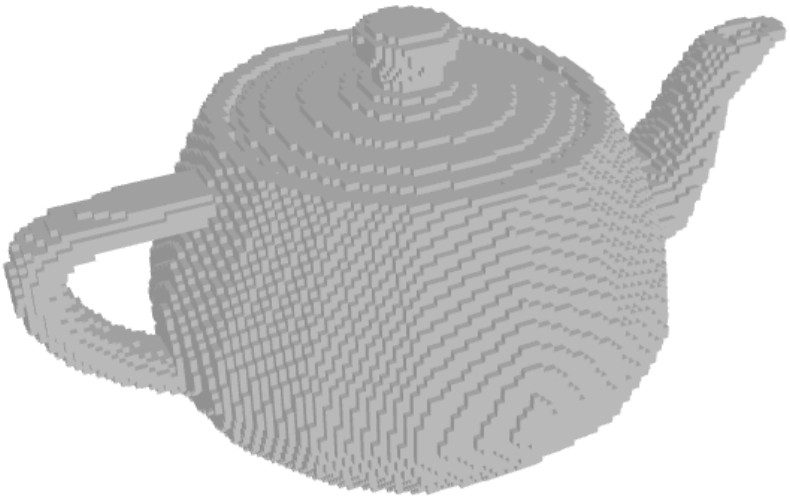
3. **Pumpkin.obj kedalaman 5:**

```
PS C:\Users\Asus\OneDrive\Documents\Tucil2_STIMA_13524009> bin/voxelize test/pumpkin.obj 5

Banyaknya voxel yang terbentuk: 4595
Banyaknya vertex yang terbentuk: 36760
Banyaknya faces yang terbentuk: 27570

Statistik node octree yang terbentuk:
1 : 8
2 : 64
3 : 448
4 : 2032
5 : 8848

Statistik node yang tidak perlu ditelusuri:
1 : 0
2 : 8
3 : 194
4 : 926
5 : 4253

Kedalaman octree: 5
Lama waktu program berjalan: 0.048 s
Path file .obj disimpan: test/pumpkin.obj_hasil.obj
```



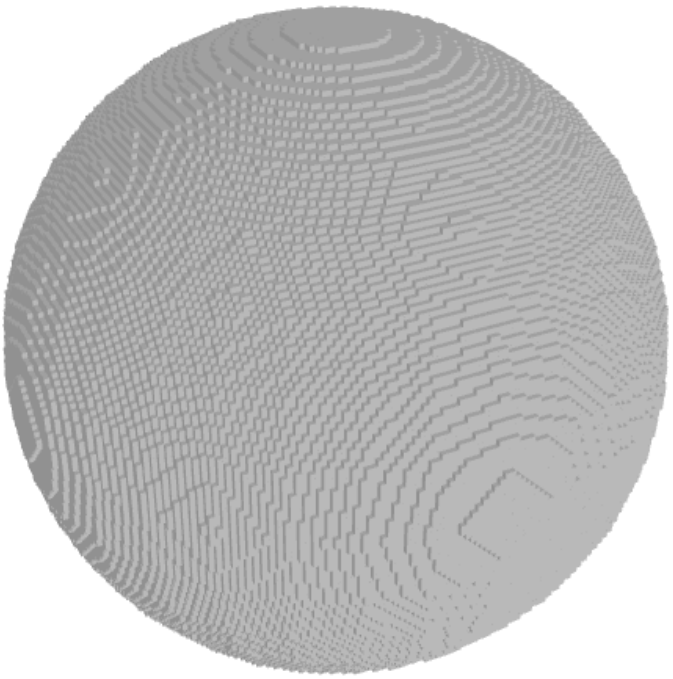
4. **Teapot.obj kedalaman 5:**

```
PS C:\Users\Asus\OneDrive\Documents\Tucil2_STIMA_13524009> bin/voxelize test/teapot.obj 5

Banyaknya voxel yang terbentuk: 1782
Banyaknya vertex yang terbentuk: 14256
Banyaknya faces yang terbentuk: 10692

Statistik node octree yang terbentuk:
1 : 8
2 : 32
3 : 160
4 : 784
5 : 3432

Statistik node yang tidak perlu ditelusuri:
1 : 4
2 : 12
3 : 62
4 : 355
5 : 1650

Kedalaman octree: 5
Lama waktu program berjalan: 0.007 s
Path file .obj disimpan: test/teapot.obj_hasil.obj
```



5. **Sphere.obj kedalaman 7:**

```
PS C:\Users\Asus\OneDrive\Documents\Tucil2_STIMA_13524009> bin/voxelize test/sphere.obj 7

Banyaknya voxel yang terbentuk: 76376
Banyaknya vertex yang terbentuk: 611008
Banyaknya faces yang terbentuk: 458256

Statistik node octree yang terbentuk:
1 : 8
2 : 64
3 : 448
4 : 2176
5 : 9280
6 : 38080
7 : 152704

Statistik node yang tidak perlu ditelusuri:
1 : 0
2 : 8
3 : 176
4 : 1016
5 : 4520
6 : 18992
7 : 76328

Kedalaman octree: 7
Lama waktu program berjalan: 0.067 s
Path file .obj disimpan: test/sphere.obj_hasil.obj

PS C:\Users\Asus\OneDrive\Documents\Tucil2_STIMA_13524009>
```



C. Lampiran

Tabel Checklist

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	✓	
2	Program berhasil dijalankan	✓	
3	Hasil konversi .obj terdiri dari kubus-kubus kecil	✓	
4	Program informasi-informasi atau report	✓	
5	Program dapat menerima masukan .obj, dan mengembalikan output	✓	
6	[Bonus] Program menggunakan *concurrency*		✓
7	[Bonus] Program dapat menampilkan visualisasi .obj		✓

Pranala Repository:

https://github.com/MikhaelBenrael/Tucil2_13524009.git

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.

Mikhael Benrael Tampubolon